

Lab 2: Basic SQL

Objectives:

- Explore the data and structure of a relational database.
- Understand the effect of NULL values in a database.
- Be able to use arithmetic logical operators in a query.
- Be able to use special functions in a query.
- Use arithmetic calculation in the select clause.
- Understand and apply the uses of the order by clause.
- Recognize primary and foreign key constraints in tables.
- Understand how key constraints maintain referential integrity.
- Apply the LIKE operator and wildcard characters in a query.
- Able to write an SQL statement to create a table.
- Write a query that accesses more than one table.

You will use the **projects_database** you built from the previous lab. There are 15 questions for you to submit to the “Lab 2: Basic SQL” assignment on gradescope. As before, each sql statement you write in response to a question is submitted in a separate .sql file. For example, the file for question 1 would be Q1.sql.

Explore the database:

It’s always a good idea to explore the data and structure of a database before you begin working with it. These two psql shell commands are useful:

View the tables: `projects_database=# \dt`

View the structure of a table: `projects_database=# \d tablename`

Observe what data is stored in this database: what entities or events do the tables store data about? How are the tables related to each other?

A bit about NULL values:

You will encounter NULL values when working with databases (of all kinds). It is very important to have an idea of how NULL values are treated by SQL as they can lead to unexpected results.

To investigate, write and execute a query to retrieve all contents of the Product table.

Do you see any “empty” places in the return? Look at projects_database.sql file and note how that value ended up being empty.

What do NULL values represent?

Think about how you would interpret NULL values if you were analyzing the data, such as performing basic statistics like mean, median, range, std dev, ect. Would you include these in your results? If you did not, would you have to mention how many instances of NULL values you removed from the data set? (answer: yes!)

What is the SQL operator that detects NULLs? Note: you can’t use the equality operator.

Selection of rows using logical operators

Q1: Write a query to retrieve all products whose price is strictly greater than \$20 but less than \$50. You should see 2 rows in the result. Select all columns. Submit the query in file Q1.sql.

Q2: Write a query to retrieve all products whose price is less than \$20 or greater than or equal to \$20. Select all columns. Submit the query in file Q2.sql.

Does your result set contain all products? If you have a Boolean condition, such as “price=20”, “price <20”, etc., is the condition satisfied when the price is NULL?

Q3: Detecting NULL values in a query. Write a query that selects all rows of the Product table that do not include null values for price. Select all columns. Submit the query in file Q3.sql.

Arithmetic operations, special functions.

Q4: The price listed in the Product table is price before tax. The sales tax in MA is 6.25%. Write a query that retrieves the list of product names and their total price after tax is applied as the column “total” (use an alias).

To do this, you will need to multiply the price column with the appropriate value to add to the before tax price in the select clause and assign it to an alias ‘total’.

For example, if price = 50.00, then after tax the price would be 53.13 rounded to two decimal places.

Note: you can try this out with the statement:

```
SELECT (your calculation with hard-coded numbers);
```

to see if your calculation is correct (you will see a lot of decimal places because you haven’t rounded yet). Then try the select using the price column instead of hard-coded numbers.

Next, use the ROUND function to round the after tax total to 2 decimal places. Note: in postgres, you need to cast the result of the after tax calculation to a numeric type using the CAST function (yes, you have to do this even though price is a double precision type).

This creates a nested set of function calls on the after tax calculation. To develop the query, start with the select clause using just the price after tax calculation. After that is correct, add the CAST to numeric and execute. When that is correct, add the outer call to ROUND:

```
ROUND( CAST ( exp as numeric), 2)
```

For reference (also lookup online if needed):

CAST (expression AS target_type) for this query, target_type is numeric.

ROUND (source , n) for this query, let n be 2.

Submit the query in file Q4.sql.

Note in the result the treatment of the NULL values. When you multiplied the NULL price value with a number, what was the result?

Ordering results

Write queries for the following:

Q5: Order all products alphabetically by name. Select all columns. Submit the query in file Q5.sql.

Q6: Order products from the most expensive to the least expensive, not including any rows that have a null value for price. Select all columns. Submit the query in file Q6.sql.

Q7: Order all products alphabetically by manufacturer, and then by descending price (for products of the same manufacturer). Do **not** filter out rows that have a null value for price. Select all columns. Submit the query in file Q7.sql.

Data changes and key constraints

Q8: Write an INSERT statement that adds a new product to the product table with these values:

name: Analyzer

price: 120.00

category: Industry

manufacturer: Hitachi

Submit the query in file Q8.sql.

Q9: Write an INSERT statement that ***violates the primary key constraint*** on the Employee table. Review what a primary key constraint guarantees. Note that an incorrect data type violates the column constraint but not a key constraint. Note that this question is asking you to write a statement that will trigger an error. The purpose is to demonstrate how the database enforces constraints to help maintain data integrity.

Submit the query in file Q9.sql.

Q10: Write a DELETE statement that violates the foreign key constraint on the product table. First, check the foreign key constraint that is defined on the product table by executing `\d product` in psql. Check the data in the product table to find a value that would help you write a delete statement (on another table) that will cause an error because it would ***violate this foreign key constraint***.

Note that this question is asking you to write a statement that will trigger an error. The purpose is to demonstrate how the database enforces constraints to help maintain data integrity.

Submit the query in file Q10.sql.

More Queries and Joins

Write the following SQL queries.

Q11: Retrieve the names of American companies that manufacture products in the 'Gadgets' category. Make sure to not return the same company name multiple times. Submit the query in file Q11.sql.

Q12: Retrieve all columns from the product table with the string 'Gizmo' in their name. Submit the query in file Q12.sql.

Q13: Retrieve the list of all unique product categories. Return only the category column. Submit the query in file Q13.sql.

Q14: Find all the countries that produce products in the 'Gadgets' category. Make sure to not return the same country name multiple times. Submit the query in file Q14.sql.

Q15: Retrieve the names of all projects that Alice is assigned to. Your query should only select the project names and should not repeat the same project multiple times. Submit the query in file Q15.sql.

If you are done with the assignment, you can test your skills solving the SQL Murder Mystery:
<https://mystery.knightlab.com/>